МИНИСТЕРСТВО НАУКИ И ВЫСШЕГО ОБРАЗОВАНИЯ РОССИЙСКОЙ ФЕДЕРАЦИИ

Федеральное государственное автономное образовательное учреждение высшего образования «Санкт-Петербургский политехнический университет Петра Великого»

Институт компьютерных наук и кибербезопасности

Высшая школа технологий и искусственного интеллекта

Направление: 02.03.01 Математика и компьютерные науки

Отчет по курсовому проекту по дисциплине "Функциональное программирование" Реализация проекта на языке Haslell по теме "Визуализатор алгоритмов на графах"

Отчет выполнил
студент группы 5130201/40001 _____ Костина А.В.
Отчет принял
преподаватель _____ Моторин Д.Е.

«____»_____ 2026г.

Санкт-Петербург, 2026

Содержание

Техническое задание **3**

Введение **12**

1 Постановка задач **13**

2 Особенности реализации **14**
2.1 Описание разработки проекта 14
2.2 Описание технических особенностей реализации проекта 14

3 Примеры работы программы **15**

4 Руководство по эксплуатации для пользователя **16**

Заключение **17**

Список литературы **18**

Приложение **19**
A.1 19

Техническое задание

1. Основание для выполнения работы

Работа выполняется в соответствии с учебным планом по дисциплине «Функциональное программирование» в рамках курсового проектирования.

2. Исполнитель и соисполнители работы

Исполнитель: студент группы 5130201/40001
Соисполнители: отсутствуют

3. Цели, задачи и исходные данные

a. Цель работы

Целью работы является разработка консольного приложения на языке Haskell для визуализации и пошагового анализа алгоритмов на графах. Приложение должно позволять создавать и редактировать графы, выполнять алгоритмы с возможностью сохранения каждого шага в PNG-файл и предоставлять детальную информацию о выполняемых операциях.

b. Задачи работы

В процессе выполнения работы требуется решить следующие задачи:

- **Чтение и запись файлов:**
  - реализовать импорт графа из текстового файла;
  - реализовать экспорт графа в текстовый файл;
  - реализовать сохранение визуализаций графа и промежуточных состояний алгоритмов в PNG-файлы.
- **Обработка информации математическими функциями:**
  - разработать структуры данных для представления ориентированных и неориентированных взвешенных графов;
  - реализовать алгоритмы: поиск в глубину, поиск в ширину, алгоритм Дейкстры;
  - реализовать вычисление характеристик графа: степени вершин, связность, компоненты связности.

- **Взаимодействие с пользователем через консоль:**
  - разработать систему меню для управления программой;
  - реализовать режим ручного создания и редактирования графа;
  - обеспечить ввод параметров алгоритмов и просмотр результатов.
- **Логирование действий пользователя:**
  - реализовать запись в лог-файл всех значимых действий;
  - добавить временные метки и уровни логирования.
- **Обработка ошибок:**
  - обеспечить корректную обработку ошибок ввода/вывода;
  - обрабатывать некорректный пользовательский ввод;
  - обрабатывать ошибки выполнения алгоритмов.

c. Исходные данные

Исходными данными для проведения работы являются:

- текстовые файлы с описанием графов;
- параметры, вводимые пользователем в консоли;
- библиотеки языка Haskell.

4. Основное содержание работы

a. Разработка проекта

Разработка проекта включает:

- проектирование архитектуры и системы типов;
- реализацию модулей для работы с графами, алгоритмами, визуализацией, вводом/выводом, логированием и обработкой ошибок;
- написание тестов с использованием QuickCheck;
- отладку и тестирование программы.

b. Отчетная документация

Подготовка отчета, включающего титульный лист, содержание, ТЗ, введение, описание разработки, примеры работы, руководство пользователя, заключение, список литературы и приложения.

5. Основные требования к выполнению работы

a. Назначение разрабатываемого программного комплекса

Программный комплекс предназначен для образовательных целей — демонстрации работы алгоритмов на графах с возможностью пошагового просмотра и визуализации. Может использоваться студентами при изучении теории графов и алгоритмов.

b. Состав разрабатываемого программного комплекса

- исполняемый модуль;
- модули исходного кода на Haskell;
- тестовые файлы с примерами графов;
- документация пользователя;
- файлы логов и визуализаций.

c. Технические характеристики

- Язык реализации: Haskell;
- Система сборки: Stack;
- Операционная система: Windows;
- Взаимодействие: консольный ввод/вывод;
- Визуализация: генерация PNG-файлов с использованием библиотеки `graphite`.

d. Функциональные требования

d.1. Работа с графом

- Создание нового графа с выбором типа (ориентированный/неориентированный).
- Добавление вершины: команда `addv <номер>`.
- Удаление вершины: команда `delv <номер>` (удаляет все связанные рёбра).
- Добавление ребра: команда `adde <u> <v> [вес]`, где вес по умолчанию = 1.
- Удаление ребра: команда `dele <u> <v>`.
- Изменение веса ребра: команда `setw <u> <v> <новый вес>`.
- Просмотр текущего состояния графа в текстовом виде.
- Очистка графа.

d.2. Импорт и экспорт файлов

- Импорт графа из текстового файла формата:

```
n m
u1 v1 w1
u2 v2 w2
...
```

  где n — количество вершин, m — количество рёбер.

- Экспорт графа в указанный формат.
- Проверка корректности файла при импорте.

d.3. Алгоритмы на графах

- **Поиск в глубину (DFS):**
  - запуск от указанной стартовой вершины;
  - отображение порядка обхода вершин;
  - пошаговый режим с подсветкой текущей вершины и стека вызовов.
- **Поиск в ширину (BFS):**

  - запуск от указанной стартовой вершины;
  - отображение порядка обхода и расстояний до вершин;
  - пошаговый режим с подсветкой текущей вершины и очереди.

- **Алгоритм Дейкстры:**
  - запуск от указанной стартовой вершины;
  - возможность указать целевую вершину;
  - отображение кратчайших расстояний до всех вершин;
  - восстановление кратчайшего пути;
  - пошаговый режим с подсветкой текущей обрабатываемой вершины и таблицы расстояний.

d.4. Характеристики графа

- Вычисление степени вершины (для ориентированных — входящей и исходящей).
- Проверка графа на связность.
- Нахождение всех компонент связности.
- Проверка наличия пути между двумя вершинами.

d.5. Визуализация

- Сохранение текущего состояния графа в PNG-файл.
- Сохранение промежуточных состояний алгоритма в серию PNG-файлов (`step_001.png`, `step_002.png`, ...).
- Цветовое кодирование состояний:
  - непосещённая вершина — белый фон, чёрная граница;
  - текущая вершина — красный фон;
  - посещённая вершина — зелёный фон;
  - вершина в очереди/стеке — жёлтый фон;
  - текущее ребро — красная линия;
  - пройденное ребро — зелёная линия;
  - непройденное ребро — серая линия.
- Подписи вершин (номера) и рёбер (веса).

d.6. Консольный интерфейс

Главное меню:

```
========== ГЛАВНОЕ МЕНЮ ==========
1. Управление файлами
2. Редактирование графа
3. Выполнение алгоритмов
4. Визуализация
5. Характеристики графа
6. Просмотр лога
7. Выход
Ваш выбор:
```

Меню "Управление файлами":

- Создать новый граф (с выбором типа)
- Загрузить граф из файла
- Сохранить граф в файл
- Сохранить визуализацию текущего графа
- Назад

Меню "Редактирование графа":

- Показать текущий граф
- Добавить вершину
- Удалить вершину
- Добавить ребро
- Удалить ребро
- Изменить вес ребра
- Очистить граф
- Назад

Меню "Выполнение алгоритмов":

- Поиск в глубину (DFS)
  - Ввести стартовую вершину
  - Запустить (обычный/пошаговый)

  - Показать результат
- Поиск в ширину (BFS)
  - Ввести стартовую вершину
  - Запустить (обычный/пошаговый)
  - Показать результат
- Алгоритм Дейкстры
  - Ввести стартовую вершину
  - Ввести целевую вершину (опционально)
  - Запустить (обычный/пошаговый)
  - Показать результат
  - Восстановить путь
- Назад

Меню "Визуализация":

- Сохранить текущее состояние графа
- Сохранить все шаги последнего алгоритма
- Выбрать цветовую схему
- Назад

Меню "Характеристики графа":

- Показать основную информацию (число вершин, рёбер, тип)
- Вычислить степень вершины
- Проверить связность
- Найти компоненты связности
- Проверить наличие пути между вершинами
- Назад

Меню "Просмотр лога":

- Показать последние N записей
- Показать все записи
- Очистить лог
- Назад

d.7. Логирование

- Файл лога: `graph_app.log`
- Формат записи: `[YYYY-MM-DD HH:MM:SS] [LEVEL] Сообщение`
- Уровни логирования: INFO, WARNING, ERROR
- Логируемые события:
  - запуск и завершение программы;
  - создание, загрузка, сохранение графа;
  - добавление/удаление вершин и рёбер;
  - запуск алгоритмов с указанными параметрами;
  - результаты выполнения алгоритмов;
  - ошибки при работе с файлами;
  - ошибки пользовательского ввода.

d.8. Обработка ошибок

- **Ошибки ввода-вывода:**
  - файл не найден — вывод сообщения "Файл не найден. Проверьте путь и имя файла";
  - некорректный формат файла — указание строки, где произошла ошибка;
  - ошибка при сохранении — вывод причины ошибки.
- **Ошибки пользовательского ввода:**
  - ввод нечисловых значений — повтор запроса с сообщением "Ожидается число";
  - указание несуществующей вершины — вывод списка существующих вершин;
  - добавление уже существующего ребра — предупреждение;
  - отрицательные веса — сообщение о недопустимости.
- **Ошибки алгоритмов:**
  - отсутствие пути между вершинами — сообщение "Путь не найден";
  - отрицательные веса в алгоритме Дейкстры — предупреждение о некорректной работе;
  - пустой граф — сообщение "Граф не содержит вершин".

е. Стандартные средства

При разработке должны использоваться типовые средства языка Haskell и существующие библиотечные решения:

- `containers` (Data.Map, Data.Set, Data.Sequence) — для представления графов и структур данных алгоритмов;
- `graphite` — для визуализации графов и сохранения в PNG;
- `QuickCheck` — для тестирования свойств;
- `directory` — для работы с файловой системой;
- `time` — для формирования временных меток в логах.

6. Этапы и сроки выполнения работы

Диаграмма Ганта

Рис. 1: Диаграмма Ганта этапов работ

7. Результаты работы

Введение

1 Постановка задач

2 Особенности реализации

2.1 Описание разработки проекта

2.2 Описание технических особенностей реализации проекта

3 Примеры работы программы

4 Руководство по эксплуатации для пользователя

Заключение

Список литературы

[1]

Приложение

A.1

```
1
2
```